

Atividade de AEDS III - Tabelas hash

Objetivo

Uma grande empresa de segurança cibernética precisa processar bilhões de registros de acessos web. Cada registro contém um endereço IP de origem. Para detectar possíveis ataques de negação de serviço (DoS), eles precisam rastrear a **frequência de requisições** de cada IP em tempo real. Mas o tempo de processamento está tornando essa operação inviável. Ajude a empresa a melhorar o código para tornar a solução robusta.

Instruções

1. Interprete, compile, execute o código fornecido, criando as funções necessárias para executá-lo. Identifique como foi implementada a **tabela hash** e quais estruturas foram utilizadas.

```
C/C++
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define TAMANHO_TABELA 1009

typedef struct Node {
    char ip[16];
    int frequencia;
    struct Node* proximo;
} Node;

Node* tabela[TAMANHO_TABELA];

unsigned int funcao_hash(char* ip) {
    unsigned int hash = 0;
    // usa caracteres ASCII do IP
    for (int i = 0; ip[i] != '\0'; i++) {
        hash += ip[i];
    }
    return hash % TAMANHO_TABELA;
}

void inserir(char* ip) {
    unsigned int indice = funcao_hash(ip);
    Node* atual = tabela[indice];
```

```

while (atual != NULL) {
    if (strcmp(atual->ip, ip) == 0) {
        atual->frequencia++;
        return;
    }
    atual = atual->proximo;
}

// aloca memória para novo nó
Node* novo = (Node*)malloc(sizeof(Node));
if (novo == NULL) {
    printf("Erro: Falha na alocação de memória.\n");
    exit(1);
}

// dados do novo nó
strcpy(novo->ip, ip);
novo->frequencia = 1;
novo->proximo = tabela[indice];
// tabela tem novo nó como referência
tabela[indice] = novo;
}

```

2. Use o comando abaixo em um terminal linux para gerar um arquivo de entrada

```

Shell
for i in {1..500000}; do echo "192.168.$((RANDOM%254+1)).$((RANDOM%254+1))"; done > dados_ips.txt

```

3. Implementar uma função auxiliar no código: `void imprimir_estatisticas()`.

Ela deve calcular e exibir:

- Total de **IPs únicos** na tabela hash.
 - O **Fator de Carga** ($\alpha = \frac{n}{m}$, onde **n** é o número de elementos e **m** o tamanho da tabela).
 - O comprimento da **maior lista** da tabela.
 - O número de posições da tabela que ficaram completamente **vazias**.
 - **Tempo de execução em milissegundos**.
4. Altere a implementação do hash para atingir o **menor tempo de execução possível**. O que deve ser feito no código para torná-lo mais rápido considerando as estatísticas obtidas? Considere as seguintes variáveis:
- Função hash.
 - Tabela hash.
 - Número de colisões.
 - Tratamento das colisões.